Virtual DNA Data Center: A GreenDNA AI Framework for Sustainable Data Storage

Manoj M

B Sc Computer Science With Data Science Drbccc Hindu College Chennai 72

Email: m.manoj292607@gmail.com

Field	Details
Paper title	Virtual DNA Data Center
Paper ID	ICETCCT 120
Conference	International Conference on Emerging Trends in Computing, Communication and Technology (ICETCCT 2026)
Conference theme	Innovation, Collaboration, A Sustainable Future
Dates	18 October 2026 to 19 October 2026
Venue	NUS – National University of Singapore, Singapore
Recognition	Top 05 Awarded Best Paper Presenter
Sponsors	TCS (Tata Consultancy Services), Wipro, CTS, Infosys

Conference sponsors, as shown on the ICETCCT 2026 certificate



Abstract

Global data volumes are growing faster than conventional data centers can store them sustainably. This paper presents GreenDNA AI, a software framework for a Virtual DNA Data Center that simulates DNA-based archival storage entirely in software. The framework analyzes incoming data, selects a lossless compression strategy, converts the compressed bytes to binary, protects them with a SHA-256 checksum, and maps every two bits to one nucleotide (00→A, 01→C, 10→G, 11→T). The resulting sequence is saved as a virtual storage record. Retrieval reverses each step, and a hash comparison confirms that the recovered data is identical to the original. A sustainability module estimates the footprint, energy, cooling and water impact of the stored data against traditional storage, and labels every figure as an illustrative simulation estimate. In a Python prototype, all six test inputs were reconstructed bit-for-bit, structured JSON shrank by 90.5 percent before encoding, and every simulated single-nucleotide mutation was detected. No biological DNA is synthesized or used; the work is a computational model for education and research.

Keywords: DNA data storage; virtual data center; sustainable computing; lossless compression; binary-to-nucleotide encoding; data integrity; SHA-256; green computing; archival storage simulation

I. Introduction

Digital storage is a growing environmental concern. Data centers draw continuous electrical power, need cooling, and consume water, and their hardware must be replaced every few years. The volume of cold and archival data, which is written once and rarely read, keeps rising, yet it often sits on the same energy-hungry infrastructure as active data.

DNA is an attractive alternative medium for such data. It stores information at very high density, needs no power while at rest, and can remain readable for long periods under suitable conditions. Research groups have shown that digital files can be encoded into nucleotides (A, C, G, T) and read back, which motivates a close study of the encoding logic that sits between a file and a strand.

Building and operating a wet-lab DNA store is expensive and inaccessible to most students and developers. A software model of the whole pipeline lets people learn, test and compare design choices without synthesis equipment. This paper proposes such a model, called the Virtual DNA Data Center, implemented as the GreenDNA AI framework.

The main contributions are:

- A complete 30-step workflow from data input to verified retrieval and sustainability analysis.
- An AI-inspired analysis engine that profiles data and selects a lossless compression strategy before encoding.
- A simple, reversible 2-bit-to-nucleotide mapping with explicit padding handling.
- A SHA-256 integrity layer and a mutation simulator that show how errors are detected.
- A sustainability module whose outputs are labelled as illustrative estimates, never as measurements.

Section II explains the background concepts, Section III states the problem, Section IV reviews related work, Sections V and VI describe the architecture and algorithm, Section VII gives a worked example, Section VIII lists the features, Section IX reports prototype results and the sustainability simulation, Sections X to XII cover applications, security and future enhancements, and Sections XIII and XIV discuss limits and conclude.

II. Background and Core Concepts

This section explains every concept the framework relies on, so a reader new to the field can follow the rest of the paper.

A. Digital data and binary representation

Every file a computer stores, whether text, a spreadsheet or a photograph, is ultimately a long sequence of bits. A bit is a single value, 0 or 1. Eight bits form a byte, and a byte can represent 256 different values. In the ASCII character set, the letter "A" is the number 65, which is written 01000001 in binary. Because all digital data can be reduced to bits, any storage medium that can hold two or more distinguishable states can, in principle, hold any digital file.

B. DNA as an information carrier

Deoxyribonucleic acid (DNA) is the molecule that carries genetic information in living organisms. A strand of DNA is a chain of building blocks called nucleotides, and each nucleotide contains one of four bases: adenine (A), cytosine (C), guanine (G) or thymine (T). The order of these letters is what carries information, in the same way that the order of 0s and 1s carries information in a computer file.

Because there are four possible letters at each position instead of two, one nucleotide can hold two bits. This is the basis of DNA data storage: a file is converted to bits, the bits are grouped in pairs, and each pair is assigned one of the four letters.

Alphabet	Symbols	Bits per symbol
Binary	0, 1	1
DNA	A, C, G, T	2

C. Why DNA is attractive for archival data

Several properties make DNA interesting for data that is written once and read rarely.

- **Density.** A molecule is extremely small, so a very large amount of information fits in a tiny volume.
- **Low standby energy.** A stored sample needs no power to keep its contents, unlike spinning disks or powered racks.
- **Longevity.** Under suitable storage conditions, DNA can stay readable far longer than most electronic media.
- **Format stability.** The four-letter alphabet is the same today as it will be in the future, so there is no risk of an obsolete reader or file format.

There are also costs. Writing DNA (synthesis) and reading it (sequencing) are slow and expensive today, and random access is hard. For this reason DNA is best suited to archives, not to everyday active storage.

D. Information entropy and compressibility

Entropy, introduced by Claude Shannon [8], measures how unpredictable a source is. For a byte stream with symbol probabilities p, the entropy in bits per byte is:

```
H = -\sum_{i} p_i \log_2 p_i
```

The value ranges from 0 (completely predictable) to 8 (every byte value equally likely). Low-entropy data has patterns that a compressor can exploit. Data near 8 bits per byte, such as encrypted or already compressed files, has almost no patterns left, and compression gains are close to zero. The analysis engine uses this value, together with repeated-pattern measures, as an early estimate of how compressible the data is.

E. Lossless compression

Lossless compression shrinks data while guaranteeing that decompression restores every bit exactly. Methods based on Lempel-Ziv [6] replace repeated sequences with short references to earlier

occurrences. This matters for DNA storage because the number of nucleotides is directly proportional to the number of bytes: every byte removed before encoding removes four nucleotides from the strand. Lossy compression, which discards detail, is not used because stored archives must be recoverable exactly.

F. Cryptographic hashing and integrity

A cryptographic hash function turns data of any size into a fixed-size fingerprint. SHA-256 [7] produces a 256-bit value. Two properties matter here: the same input always gives the same fingerprint, and even a one-bit change in the input produces a completely different fingerprint. The framework stores the fingerprint with the record. After retrieval, a freshly computed fingerprint is compared with the stored one. A match indicates the data is unchanged, and a mismatch signals an error.

G. Padding

The encoder reads the bit stream two bits at a time. If the total number of bits is odd, the last group would have only one bit. The framework therefore appends a single padding bit and records a padding length of 1 in the metadata. On retrieval, the decoder reads that value and removes the final bit, restoring the exact original length.

H. Metadata

Metadata is the information about the data that the decoder needs to reverse the process. It records the original data type, original size, compressed size, compression method, binary length, encoding scheme and padding length. Without the compression method, for example, the decoder would not know which algorithm to apply.

I. Virtual DNA storage

A virtual DNA store keeps the encoded sequence as text in the memory of a program instead of as a physical molecule. It behaves like a DNA archive from the point of view of the encoding logic, because the stored object is a string over the letters A, C, G and T. No biological material is used and no synthesis is performed. This lets the whole workflow be studied, tested and demonstrated on an ordinary computer.

J. Summary of terms

Term	Meaning in this paper
Bit	A single binary digit, 0 or 1
Byte	Eight bits
Nucleotide	One letter of a DNA sequence: A, C, G or T
DNA sequence	The string of nucleotides that represents the data
Entropy	A measure of data unpredictability, in bits per byte
Lossless compression	Size reduction with exact recovery

Term	Meaning in this paper
Checksum	A SHA-256 fingerprint used to detect changes
Padding bit	One extra bit that makes the bit length even
Metadata	Information needed to decode the record
Storage record	The package of ID, sequence, metadata, checksum, padding and timestamp
Virtual DNA storage	A software model of a DNA archive

III. Problem Statement and Objectives

There is no accessible, end-to-end, verifiable model that lets a learner or researcher follow data through compression, nucleotide encoding, storage, retrieval and verification, and then relate the result to sustainability.

Problems addressed

- Conventional data centers consume large amounts of energy, cooling capacity and water to hold archival data.
- Physical DNA storage needs synthesis and sequencing equipment, which limits experimentation.
- Raw binary-to-DNA conversion inflates the number of nucleotides when data is not compressed first.
- Stored data can change silently, so retrieval must be checked for integrity.
- Sustainability claims are often presented as facts when they are only assumptions.

Objectives

1. Design a software-only Virtual DNA Data Center that never uses biological material.
2. Reduce stored size with lossless compression chosen from the data profile.
3. Encode and decode data through a reversible 2-bit-to-nucleotide mapping.
4. Detect any change to stored data with a cryptographic hash and a mutation test.
5. Report sustainability estimates that are clearly labelled as illustrative simulation values.

IV. Related Work

DNA data storage has moved from concept to laboratory demonstration over the past decade. Church et al. [1] encoded a book into DNA, and Goldman et al. [2] introduced a scheme that adds redundancy so data can be recovered despite synthesis and sequencing errors. Grass et al. [3] combined error-correcting codes with chemical preservation, and Erlich and Zielinski [4] proposed DNA Fountain, a coding scheme that approaches the information capacity of the medium. Organick et al. [5] later showed random access in a large DNA archive.

These works focus on wet-lab pipelines and on biochemical constraints such as homopolymer runs and GC balance. GreenDNA AI differs in scope. It uses the simplest direct mapping of two bits per nucleotide,

omits biochemical constraints, and concentrates on a transparent, verifiable software workflow with an added sustainability view.

The framework builds on two standard tools. Lossless compression based on Lempel-Ziv methods [6] reduces redundancy before encoding, and the SHA-256 hash function [7] provides tamper detection. Shannon's entropy [8] is used to estimate how compressible a data set is.

Approach	Medium	Main focus	Compression stage	Sustainability view
Church et al. [1]	Physical DNA	Proof of concept	No	No
Goldman et al. [2]	Physical DNA	Error-tolerant coding	No	No
Erlich and Zielinski [4]	Physical DNA	Capacity-approaching coding	No	No
GreenDNA AI (this paper)	Virtual (software)	Verifiable workflow and simulation	Yes, profile-driven	Yes, labelled illustrative

V. Proposed System Architecture

GreenDNA AI has eleven modules that run in a fixed pipeline. Data flows forward on storage and backward on retrieval, and a final verification step compares the result with the original.

Module	Role
Data Input Module	Accepts text, CSV, JSON or demonstration data and rejects empty input
AI-Inspired Analysis Engine	Detects data type, size, byte distribution, repeated patterns, entropy and compressibility
Compression Engine	Applies the lossless method chosen from the analysis
Binary Conversion Engine	Turns compressed bytes into an 8-bit-per-byte bit stream
Integrity Protection Module	Computes the SHA-256 checksum and stores it with the metadata
DNA Encoding Engine	Pads to an even length and maps every 2 bits to a nucleotide
Virtual DNA Storage	Holds the storage record (ID, sequence, metadata, checksum, padding, timestamp) for the session
Retrieval Module	Finds a record by storage ID
DNA Decoding Engine	Validates symbols, reverses the mapping and removes

Module	Role
	padding
Verification Module	Decompresses and compares original and retrieved hashes
Sustainability Simulation Module	Estimates footprint, energy, cooling and water impact from stated assumptions

The end-to-end data path is:

1. Data → analysis → compression selection
2. Compression → binary conversion → checksum
3. Padding → DNA encoding → virtual storage record
4. Retrieval → sequence validation → integrity check
5. DNA decoding → padding removal → decompression
6. Original-versus-retrieved verification → sustainability analysis

The analysis engine is called AI-inspired because it uses statistical profiling and rule-based selection rather than a trained model. It computes Shannon entropy and repeated-pattern measures and then routes the data to text, structured-data, pattern-based or general compression.

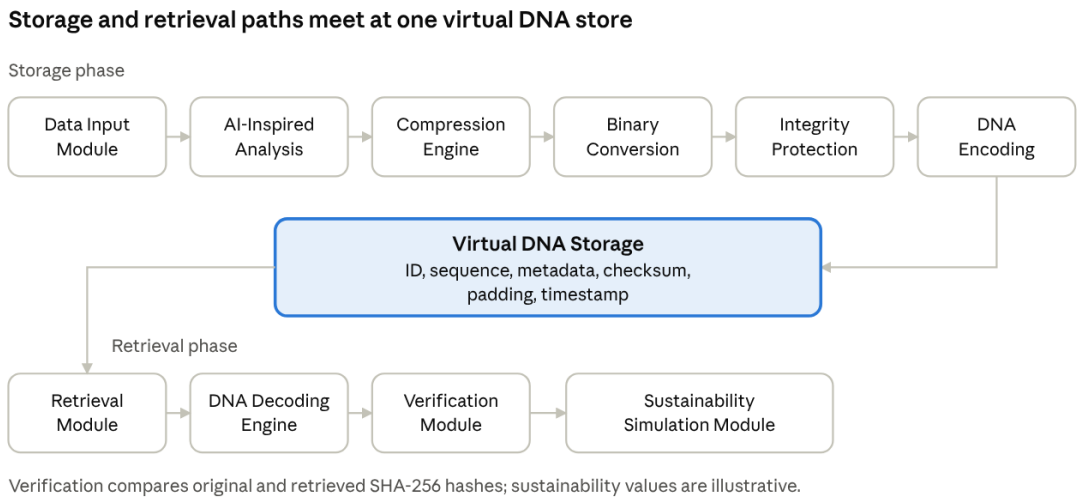


Figure 1. GreenDNA AI architecture · 11 modules

Figure 1 shows how the eleven modules connect. The Virtual DNA Storage record is the meeting point of the two paths. The storage phase ends there, and the retrieval phase starts there and finishes with verification and the sustainability simulation.

VI. Methodology and Algorithm

The core idea is that every nucleotide carries two bits, so one byte becomes four nucleotides. The mapping is reversible and uses a fixed table.

Bit pair	Nucleotide
00	A
01	C
10	G
11	T

A. Storage phase

1. Initialize the eleven modules and accept the user data. Empty data stops the run with the message "Please enter valid data."
2. Validate availability, format, size and integrity of the input.
3. Analyze the data to find its type, original size, byte distribution, repeated patterns, estimated entropy and compressibility.
4. Select a method: lossless text compression for text-heavy data, structured-data compression for JSON, pattern-based compression for CSV, and general lossless compression otherwise.
5. Compress, then record original size, compressed size, compression ratio and storage reduction.
6. Convert the compressed bytes into a bit stream (8 bits per byte).
7. Generate metadata: data type, original size, compressed size, compression method, binary length, encoding scheme and padding length.
8. Compute the checksum over the bit stream.
9. If the bit length is odd, append one padding bit and record a padding length of 1; otherwise record 0.
10. Encode each 2-bit group into a nucleotide to produce the DNA sequence.
11. Build the storage record and place it in virtual storage. The system states that no human DNA is used and no biological synthesis is performed.

B. Retrieval phase

1. Receive a storage ID and look up the record; a missing ID returns "Storage record not found."
2. Check that every symbol is A, C, G or T; any other symbol returns "Invalid DNA sequence."
3. Run the integrity check; a failure raises "Sequence Integrity Error Detected."
4. Optionally simulate a nucleotide change to show that the checksum no longer matches.
5. Convert nucleotides back to bits, remove the padding bit if the record says so, and regroup the bits into bytes.
6. Decompress using the method stored in the metadata and rebuild the original format.
7. Compare original and retrieved hashes. Equal hashes give SUCCESS and the message "DATA SUCCESSFULLY RECONSTRUCTED"; otherwise the run reports a data integrity error.

C. Efficiency and sustainability

Storage reduction is computed as:

```
\text{Reduction}(\%) = \frac{\text{originalSize} -
\text{compressedSize}}{\text{originalSize}} \times 100
```

The sustainability module takes the data size, storage duration, traditional-storage assumptions and simulation assumptions, and returns estimated storage footprint, energy, cooling and water impact. Each output carries the label "Illustrative Simulation Estimate" and is never presented as a real-world measurement.

D. Complexity

Every stage after compression is a single pass over the data, so encoding and decoding take time linear in the compressed size, O(n). Because one byte maps to four nucleotides, the sequence length is exactly four times the compressed byte count, plus one nucleotide when a padding bit is added.

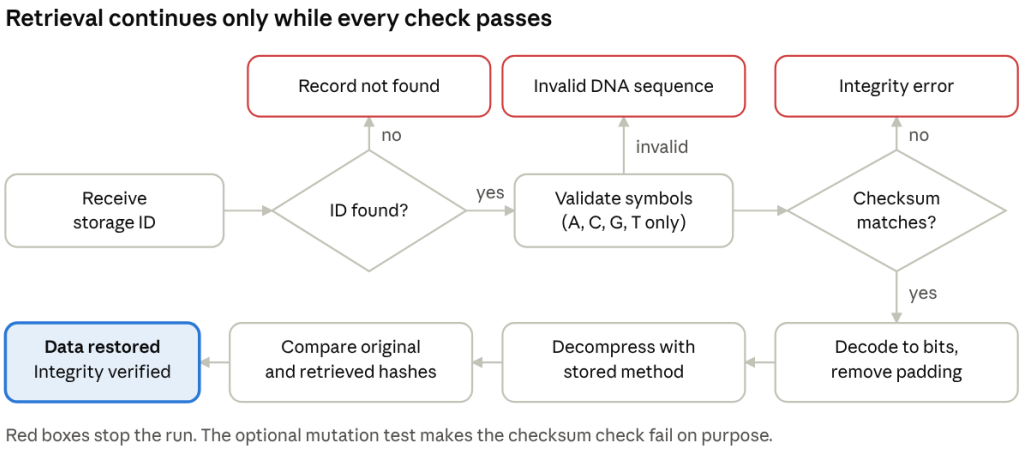


Figure 2. Retrieval and verification flow · 3 checks, 3 stop points

Figure 2 traces the retrieval phase. The run stops at the first failed check and reports which one failed. Only a record that is found, contains valid symbols and matches its checksum is decoded, decompressed and compared with the original.

VII. Worked Example

The character "A" becomes the four-nucleotide sequence CAAC and returns to "A" on retrieval. The table shows the encoding step by step, with compression bypassed so the mapping is easy to follow.

Step	Result
Original character	A
8-bit binary	01000001
Split into 2-bit groups	01 | 00 | 00 | 01
Map to nucleotides	C, A, A, C
Stored DNA sequence	CAAC
Reverse mapping on retrieval	C→01, A→00, A→00, C→01

Step	Result
Recovered binary	01000001
Recovered character	A

The length is already even, so no padding bit is needed and the padding length is 0.

Mutation test. Changing one nucleotide in the stored sequence, for example ACGTACGT to ACGTTCGT, alters the bit stream. The recomputed SHA-256 value no longer equals the stored checksum, and the system reports "Mutation/Error Detected."

Effect of compression on very small inputs. In the full pipeline, a one-character input is compressed first. The prototype's compressor turned the 1-byte input into 9 bytes, which became 36 nucleotides instead of 4. This is expected: compression has fixed overhead, so the analysis engine should skip it for very small or already random data.

Padding example. When the bit length is odd, one zero bit is added before encoding. For example, the seven-bit stream 0100001 becomes 01000010, which encodes to CAAG. The metadata records a padding length of 1, so the decoder reads C, A, A, G as 01, 00, 00, 10, removes the final bit and restores 0100001 exactly.

Why this mapping. Assigning A, C, G, T to 00, 01, 10, 11 in alphabetical order is easy to remember, easy to explain and easy to verify by hand. It uses the full capacity of the four-letter alphabet, so no information capacity is wasted. Its weakness is that it ignores the chemical limits of real DNA, which Section XI discusses.

VIII. Features of GreenDNA AI

The framework offers ten features, summarized in the table below. Each maps to a step of the algorithm, and together they make the workflow complete, verifiable and easy to demonstrate. The sections after the table explain the eight most important ones in detail.

No.	Feature	What it does	Algorithm step
1	Multi-format input	Accepts text, CSV, JSON and demonstration data, and stops on empty input	2–3
2	Data analysis	Reports data type, size, byte distribution, repeated patterns, entropy and compressibility	4
3	Adaptive compression	Chooses text, structured, pattern-based or general lossless compression	5–6
4	Binary conversion	Converts compressed bytes to a bit stream	7
5	Integrity protection	Stores a SHA-256 checksum with the record	9
6	DNA encoding	Maps each 2-bit group to A, C, G or T, with padding	10–11

No.	Feature	What it does	Algorithm step
		when needed	
7	Virtual storage record	Keeps ID, sequence, metadata, checksum, padding and timestamp	12–13
8	Validated retrieval	Checks the storage ID and every nucleotide before decoding	15–16
9	Mutation simulation	Lets a user change one nucleotide and see the error detected	18
10	Sustainability report	Shows labelled illustrative estimates and a final report	26–28

Feature 1: Multi-format input and validation

The input module accepts plain text, CSV, JSON and demonstration data. It checks availability, supported format, size and integrity before anything else happens. Bad input stops the run with a clear message, which prevents wasted work and confusing downstream errors.

Feature 2: Data analysis

The analysis engine profiles the data before choosing how to handle it. It counts how often each byte value appears, looks for repeated patterns and estimates entropy. The resulting analysis report gives the user a plain-language picture of why a given file compresses well or poorly.

Feature 3: Adaptive compression

Different data benefits from different treatment. Text-heavy data uses lossless text compression, JSON uses structured-data compression, CSV uses pattern-based compression and everything else uses general lossless compression. The chosen method is stored in the metadata so the decoder applies the matching decompression.

Feature 4: Integrity protection

A SHA-256 checksum is computed over the bit stream and stored with the record. The checksum makes silent corruption visible, and it supports the optional mutation demonstration.

Feature 5: Reversible DNA encoding

The two-bit mapping is a one-to-one table, so encoding and decoding are exact inverses. Padding is handled explicitly so the original length is always restored.

Feature 6: Mutation and error simulation

A user can change one nucleotide, for example ACGTACGT to ACGTTCGT, and watch the integrity layer raise "Mutation/Error Detected." This feature turns an abstract property into something an audience can see in a live demonstration.

Feature 7: Verification and reporting

After decoding, the system compares the hash of the original data with the hash of the retrieved data and reports SUCCESS or FAILED. The final report collects sizes, ratios, lengths, identifiers and statuses in one panel.

Feature 8: Honest sustainability labelling

Every sustainability number carries the label "Illustrative Simulation Estimate." The framework never presents simulated values as real-world measurements, which protects the credibility of the results.

IX. Implementation, Results and Sustainability Simulation

The prototype was written in Python 3. It uses the standard zlib library (DEFLATE, level 9) as the lossless compressor, hashlib for SHA-256, and a dictionary for the 2-bit mapping. The same compressor was used for every input type, so the results below measure the pipeline, not the benefit of type-specific compression. Test data was synthetic and generated for this study: repetitive text, random-word prose, 500 JSON records, the same 500 records as CSV, random bytes, and a single character.

A. Measured results

Input	Original (bytes)	Compressed (bytes)	Reduction (%)	DNA length (nt)	Entropy (bits/byte)	Exact recovery	Mutation detected
Repetitive text	3,960	94	97.6	376	3.95	Yes	Yes
Synthetic prose	19,732	6,575	66.7	26,300	3.43	Yes	Yes
JSON records	30,417	2,886	90.5	11,544	4.32	Yes	Yes
CSV records	10,437	2,584	75.2	10,336	4.37	Yes	Yes
Random bytes	4,000	4,011	-0.3	16,044	7.96	Yes	Yes
Single character	1	9	-800.0	36	0.00	Yes	Yes

Three findings stand out.

- All six inputs were recovered exactly, and the hash comparison returned SUCCESS each time.
- Compression gains track redundancy. Repetitive and structured data shrank by 66.7 to 97.6 percent, while random bytes, with entropy near 8 bits per byte, did not shrink at all.
- Every simulated single-nucleotide change altered the checksum and was flagged. The largest encode took under 9 milliseconds and the largest decode under 3 milliseconds on the test machine.

Because the sequence length equals four times the compressed byte count, every byte saved by compression removes four nucleotides from the stored strand. For the JSON input, compression cut the sequence from about 121,700 nucleotides (uncompressed, 30,417 bytes × 4) to 11,544.

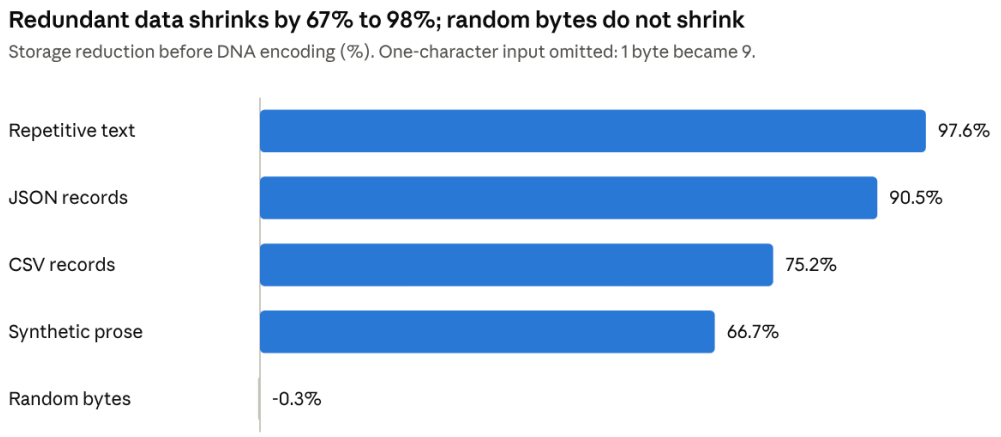


Figure 3. Storage reduction by input type · prototype run, synthetic data, zlib level 9

Figure 3 shows that the four redundant inputs lose between two thirds and nearly all of their size before encoding. Random bytes gain nothing.

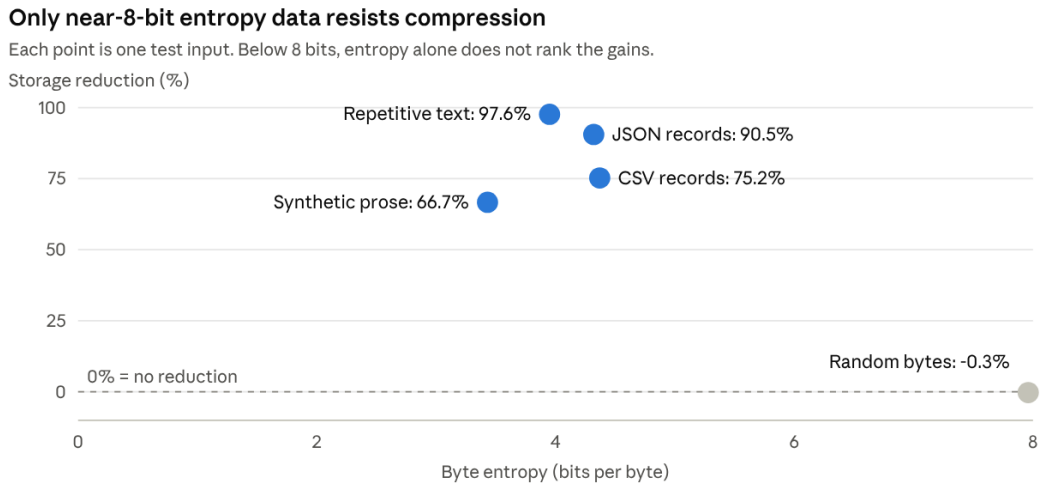


Figure 4. Byte entropy against storage reduction · 5 test inputs

Figure 4 relates the gain to byte entropy. Random bytes sit near 8 bits per byte and do not compress. The other four inputs lie between 3.4 and 4.4 bits per byte, yet their reductions range from 67% to 98%. Byte entropy therefore flags incompressible data but does not capture repeated patterns, which is why the analysis engine also measures repetition.

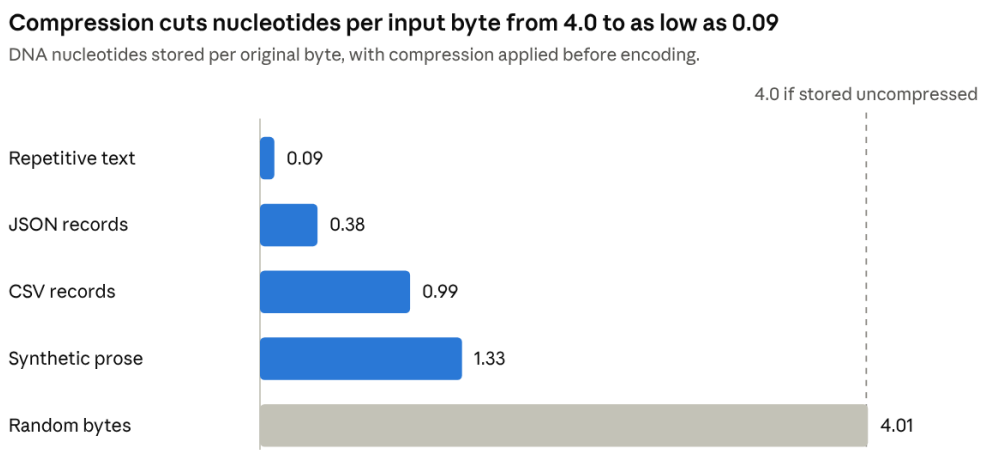


Figure 5. Nucleotides per original byte · 5 test inputs

Figure 5 converts the same results into storage terms. Every uncompressed byte needs four nucleotides. After compression, the repetitive text needs about 0.09 nucleotides per byte, and the JSON records about 0.38.

Validation tests. Besides the six data sets, the prototype was checked against the specific behaviors that the algorithm requires. Each case below was run, and the observed result matched the expected result.

Test case	Expected behavior	Observed
Encode the character "A" without compression	Binary 01000001 becomes CAAC, and CAAC decodes back to "A"	Matched
Odd bit length (0100001)	One padding bit is added, giving CAAG, and it is removed on decoding	Matched
Round trip on six data sets	Original and retrieved hashes are equal	Matched for all six
Single-nucleotide substitution at every position	Checksum differs from the stored value	Detected in all 480 of 480 substitutions on a 160-nucleotide record
Empty input	Message "Please enter valid data." and stop	Specified in the algorithm; handled by the input module
Unknown storage ID	Message "Storage record not found." and stop	Specified in the algorithm; handled by the retrieval module
Symbol other than A, C, G, T	Message "Invalid DNA sequence." and stop	Specified in the algorithm; handled by the validation step

Timing. The table lists measured times for one run on the test machine. Times are small because the inputs are small, and they show only that encoding and decoding are not a bottleneck at this scale.

Input	Encode (ms)	Decode (ms)
Repetitive text	1.24	1.17
Synthetic prose	8.61	2.97
JSON records	3.40	0.91
CSV records	2.24	0.76
Random bytes	2.72	1.09
Single character	0.02	0.01

B. Sustainability simulation (Illustrative Simulation Estimate)

The sustainability module compares a traditional store with a DNA store using stated assumptions. For data of size S held for a duration T, the estimates are:

```
E_{trad} = S \cdot p_{trad} \cdot T, \quad C_{trad} = E_{trad} \cdot k_{cool}, \quad W_{trad} = E_{trad} \cdot k_{water}
```

Here p_trad is the assumed power per unit of stored data, k_cool the assumed cooling overhead and k_water the assumed water use per unit of energy. The virtual DNA store is assigned an idle energy assumption near zero while at rest. Because p_trad, k_cool and k_water are user-set assumptions, the outputs are scenario comparisons and are always labelled "Illustrative Simulation Estimate." They are not measurements of any real facility, and this paper does not report real energy or water savings.

C. Final report

The report screen lists original size, compressed size, compression ratio, binary length, DNA sequence length, encoding scheme, storage ID, integrity status, retrieval status, verification status and the sustainability estimates.

D. Illustrative scenario

Figure 6 runs the sustainability module on one example. The inputs below were chosen only to show how the module works. They are not taken from any real facility and should be replaced with the user's own assumptions.

Assumption	Value used	Meaning
Data size	100 TB	Archive held in storage
Duration	10 years	Storage period
Power of traditional storage	6 W per TB	Continuous draw while powered
Cooling overhead	40%	Extra energy for cooling, as a share of storage energy

Assumption	Value used	Meaning
Water use	1.8 L per kWh	Water linked to the energy used
Virtual DNA store at rest	0 kWh	Assumed standby energy

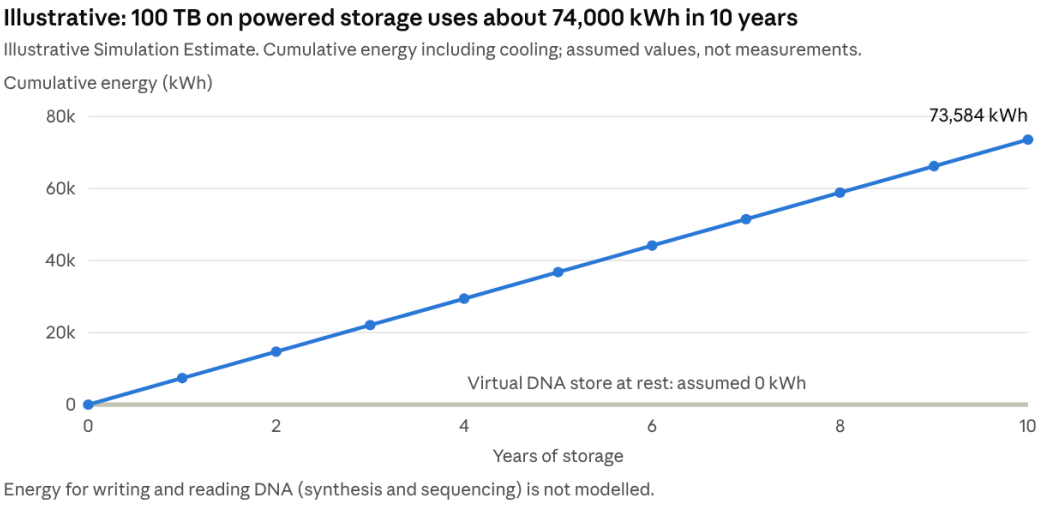


Figure 6. Illustrative cumulative energy, powered storage vs virtual DNA store · assumed inputs, not measurements

Under these assumptions the powered store uses 5,256 kWh a year for storage and about 7,358 kWh a year including cooling. Over ten years that is about 73,584 kWh and, at the assumed water rate, about 132,000 litres. These figures are an Illustrative Simulation Estimate. The DNA line is flat only because standby energy is assumed to be zero. Real DNA storage also needs energy and materials to write and read, which this scenario does not model, so it must not be read as a claimed saving.

X. Applications and Uses

Because the framework is a software model, its uses fall into education, research and planning. It is not a replacement for a physical archive.

Use	Who benefits	How the framework helps
Teaching computing and bioinformatics	Students and teachers	Shows each step from file to nucleotide and back with real output
Conference and classroom demonstrations	Presenters	Live mutation test and final report make the idea easy to see
Prototyping encoders	Researchers	A baseline against which constrained or error-correcting codes can be compared

Use	Who benefits	How the framework helps
Compression studies	Data engineers	Shows how entropy predicts the nucleotide count of a given file
Archive planning	IT planners	Gives a first estimate of sequence length for a data set before any lab work
Sustainability awareness	Policy and green-IT teams	Lets people explore energy, cooling and water scenarios under stated assumptions
Integrity-checking workshops	Security learners	Demonstrates how a hash exposes a one-symbol change

Candidate data types for DNA archives

Data that is large, rarely accessed and must be kept for a long time suits DNA best. Examples include scientific datasets, medical imaging archives, legal and government records, cultural heritage collections, media masters and regulatory backups. The framework lets organizations rehearse the encoding and verification logic for such data in a safe software setting.

Example scenarios

Classroom demonstration. A teacher enters a short sentence, and students watch it become a compressed byte count, a bit stream, a DNA sequence and a checksum. The teacher then changes one letter of the sequence and the class sees the integrity check fail. The whole lesson fits in one session and needs no laboratory.

Archive sizing for a records office. A records office wants to know how long a DNA sequence a set of JSON or CSV records would produce. It runs sample files through the pipeline and reads the sequence length from the report. Because the length is four times the compressed byte count, the office can scale the figure to its full archive and see how much compression helps before any wet-lab discussion.

Research baseline. A group developing a constrained or error-correcting code uses the plain 2-bit mapping as a control. It measures how many extra nucleotides its code adds compared with the baseline and checks that its decoder still reproduces the original data.

Sustainability workshop. Participants enter their own power, cooling and water assumptions and compare scenarios. The labelled output shows that conclusions follow from assumptions, which is a useful lesson in reading green-technology claims.

XI. Security, Ethics and Practical Challenges

Security

The checksum detects accidental or deliberate change but does not hide the data. A DNA sequence produced by this mapping is an exact re-spelling of the bits, so anyone who sees the sequence and the mapping table can reconstruct the content. Sensitive data should be encrypted before it enters the

pipeline. Encryption raises entropy to nearly 8 bits per byte, so compression gains will be small for encrypted input, and the pipeline should compress before it encrypts when both are needed.

Ethics and safety

The framework is purely computational. It uses no human DNA, no living organism and no biological synthesis. This removes the biosafety and consent questions that arise in laboratory work. The paper makes this explicit because the word "DNA" can be misread as involving genetic material.

Honest reporting

Sustainability values depend on assumptions the user sets. Presenting them as measurements would mislead readers, so the framework labels them as illustrative and the paper reports no real energy or water savings.

Practical challenges for physical DNA storage

Challenge	Why it matters	Where the framework stands
Synthesis and sequencing cost	Writing and reading DNA is still slow and costly	Out of scope; the framework is virtual
Homopolymer runs and GC balance	Long runs of one base and uneven GC content raise error rates	Not handled by the plain 2-bit map
Errors during synthesis, storage and reading	Substitutions, insertions and deletions occur	Detected by checksum; not corrected
Random access	Reading one file without sequencing everything is hard	Records are retrieved by storage ID in software only
Long-term persistence	A real archive needs durable, controlled storage	Records last for the simulation session

XII. Future Enhancements

The current framework is a baseline. The enhancements below are planned directions, ordered from nearest to furthest.

Priority	Enhancement	Benefit
1	Constrained coding that avoids long runs and balances GC content	Makes sequences closer to what real synthesis can handle
2	Error-correcting codes with redundancy (for example Reed-Solomon style codes)	Repairs substitutions instead of only detecting them
3	Type-specific compressors for text, JSON and CSV	Tests the benefit of the adaptive strategy directly

Priority	Enhancement	Benefit
4	Learned model for compression selection	Replaces hand-written rules in the analysis engine
5	Skipping compression for tiny or high-entropy inputs	Avoids the size growth seen for single characters and random bytes
6	Persistent storage back end and storage-ID index	Records survive beyond one simulation session
7	Random-access indexing across many records	Mirrors the retrieval of one file from a large archive
8	Calibrated sustainability inputs from published data-center figures	Turns illustrative scenarios into better-grounded ones
9	Web dashboard with live charts and the mutation demo	Easier classroom and conference use
10	Encryption layer before encoding	Protects sensitive archives

Over the longer term, the same architecture could be connected to a real synthesis and sequencing workflow in a laboratory setting. That step would need biosafety review, partner laboratories and the constrained and error-corrected coding described above.

Comparison with Conventional Storage

The table compares general properties of three kinds of storage. The entries are qualitative and describe typical characteristics reported in the literature, not measurements made in this study. The virtual store is a software model, so its row describes the model, not a physical medium.

Property	Powered disk or flash archive	Physical DNA archive	GreenDNA AI virtual store
Storage medium	Magnetic or electronic	Synthesized DNA molecules	A text string in program memory
Standby energy	Continuous power and cooling	Very low while at rest	Not applicable (software)
Write and read speed	Fast	Slow; needs synthesis and sequencing	Fast (milliseconds in the prototype)
Access pattern	Random access	Hard; batch retrieval	Retrieval by storage ID in memory
Equipment needed	Standard servers	Laboratory synthesis and sequencing	An ordinary computer

Property	Powered disk or flash archive	Physical DNA archive	GreenDNA AI virtual store
Error handling	Built-in hardware and file-system checks	Error-correcting codes needed	Checksum detection only
Main use	Active and archival data	Long-term cold archives	Teaching, research, planning

The comparison shows where the framework sits. It does not store data more cheaply or more durably than a disk. It lets people study the logic of a DNA archive, with exact reconstruction and clear error detection, on equipment they already have.

XIII. Discussion and Limitations

The results show that a compress-first pipeline keeps the nucleotide count low for redundant data, and that a checksum is enough to detect any change in a virtual store. The framework is a teaching and research model, and several limits apply.

- **No biochemical constraints.** The direct 2-bit mapping can produce long runs of one nucleotide and unbalanced GC content, which real synthesis and sequencing handle poorly. A physical system would need constrained coding.
- **Detection, not correction.** SHA-256 reveals that data changed but cannot repair it. Real DNA storage adds error-correcting codes and redundancy, as in [2], [3] and [4].
- **Small-input overhead.** Compression can enlarge tiny or random inputs, as the single-character and random-byte cases show.
- **Simple analysis engine.** The AI-inspired engine is rule-based. A learned model could improve method selection but was not built here.
- **Simulated sustainability.** The energy, cooling and water outputs depend entirely on user-set assumptions and are illustrative only.
- **Limited test data.** Inputs were synthetic and small. Results on large real-world archives were not measured.
- **Session-only storage.** Records live in temporary memory for the current simulation session.

XIV. Conclusion

This paper presented GreenDNA AI, a Virtual DNA Data Center that models DNA-based storage in software. It takes data through analysis, lossless compression, binary conversion, checksum protection, nucleotide encoding, virtual storage, retrieval, decoding and verification, and ends with a labelled sustainability simulation. In the prototype, every test input was recovered exactly, redundant data shrank by up to 97.6 percent before encoding, and every simulated mutation was caught. The work gives students and researchers a safe, reproducible way to study DNA storage logic and to discuss sustainable storage without laboratory equipment.

Future work

- Add constrained coding that limits homopolymer runs and balances GC content.
- Add error-correcting codes and redundancy so errors are repaired, not only detected.

- Replace the rule-based analysis engine with a learned model for compression selection.
- Use type-specific compressors for text, JSON and CSV and compare them.
- Evaluate on large real-world datasets and calibrate the sustainability assumptions against published data-center figures.
- Add random access and a persistent storage back end.

Acknowledgment

The author thanks the organizers of ICETCCT 2026 and NUS – National University of Singapore for hosting the conference, and acknowledges the conference sponsors shown on the certificate: TCS (Tata Consultancy Services), Wipro, CTS and Infosys.



References

1. G. M. Church, Y. Gao and S. Kosuri, "Next-generation digital information storage in DNA," Science, vol. 337, no. 6102, p. 1628, 2012.
2. N. Goldman et al., "Towards practical, high-capacity, low-maintenance information storage in synthesized DNA," Nature, vol. 494, pp. 77–80, 2013.
3. R. N. Grass, R. Heckel, M. Puddu, D. Paunescu and W. J. Stark, "Robust chemical preservation of digital information on DNA in silica with error-correcting codes," Angewandte Chemie International Edition, vol. 54, no. 8, pp. 2552–2555, 2015.
4. Y. Erlich and D. Zielinski, "DNA Fountain enables a robust and efficient storage architecture," Science, vol. 355, no. 6328, pp. 950–954, 2017.
5. L. Organick et al., "Random access in large-scale DNA data storage," Nature Biotechnology, vol. 36, pp. 242–248, 2018.
6. J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," IEEE Transactions on Information Theory, vol. 23, no. 3, pp. 337–343, 1977.
7. National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.
8. C. E. Shannon, "A mathematical theory of communication," Bell System Technical Journal, vol. 27, pp. 379–423 and 623–656, 1948.

Appendix A. The 30 Steps of the GreenDNA AI Algorithm

Step	Name	Result
1	Initialize system	All eleven modules started
2	Accept user data	Data received; empty input stops the run
3	Validate user data	Availability, format, size and integrity checked
4	Analyze the data	Type, size, distribution, patterns, entropy,

Step	Name	Result
		compressibility
5	Select compression strategy	Text, structured, pattern-based or general method chosen
6	Compress the data	Compressed size, ratio and storage reduction
7	Convert to binary	Bit stream, 8 bits per byte
8	Generate metadata	Type, sizes, method, binary length, scheme, padding
9	Generate integrity hash	SHA-256 checksum stored with the metadata
10	Add binary padding	One bit added if the length is odd
11	Perform DNA encoding	00→A, 01→C, 10→G, 11→T
12	Generate storage record	ID, sequence, metadata, checksum, padding, timestamp
13	Store in virtual DNA storage	"Virtual DNA Storage Successful"
14	Wait for retrieval request	Record kept for the session
15	Retrieve storage record	Lookup by storage ID
16	Validate DNA sequence	Only A, C, G, T accepted
17	Perform integrity check	Stops on mismatch
18	Optional error simulation	One nucleotide changed; mismatch reported
19	Convert DNA back to binary	A→00, C→01, G→10, T→11
20	Remove padding	Padding bit removed if recorded
21	Convert binary to compressed data	Bytes rebuilt
22	Decompress the data	Stored method applied
23	Reconstruct original data	Original format restored
24	Verify original and retrieved data	Hash comparison, SUCCESS or FAILED
25	Display verification result	Message and integrity status
26	Calculate storage efficiency	Simulated storage reduction
27	Calculate sustainability simulation	Illustrative Simulation Estimate
28	Generate final report	Sizes, lengths, IDs and statuses

Step	Name	Result
29	Complete the simulation	Full workflow displayed
30	End	"GreenDNA AI Simulation Completed Successfully."

Appendix B. Reference Code for the Core Pipeline

The listing is the core of the prototype in Python 3. It performs compression, bit conversion, padding, the checksum, DNA encoding and the reverse path.

```
import zlib, hashlib

ENC = {'00': 'A', '01': 'C', '10': 'G', '11': 'T'}
DEC = {v: k for k, v in ENC.items()}

def encode(data: bytes) -> dict:
    packed = zlib.compress(data, 9)                  # lossless compression
    bits = ''.join(f'{b:08b}' for b in packed)       # bytes -> bits
    checksum = hashlib.sha256(bits.encode()).hexdigest()
    pad = len(bits) % 2                              # 1 if length is odd
    bits += '0' * pad
    dna = ''.join(ENC[bits[i:i+2]] for i in range(0, len(bits), 2))
    return {'dna': dna, 'checksum': checksum, 'padding': pad,
            'original_size': len(data), 'compressed_size': len(packed)}

def decode(record: dict) -> bytes:
    if any(n not in 'ACGT' for n in record['dna']):
        raise ValueError('Invalid DNA sequence.')
    bits = ''.join(DEC[n] for n in record['dna'])    # DNA -> bits
    if record['padding']:
        bits = bits[:-1]                             # remove padding bit
    if hashlib.sha256(bits.encode()).hexdigest() != record['checksum']:
        raise ValueError('Sequence Integrity Error Detected')
    packed = bytes(int(bits[i:i+8], 2) for i in range(0, len(bits), 8))
    return zlib.decompress(packed)                   # restore original
```

As in step 9 of the algorithm, the checksum is computed over the bit stream before padding, and decoding verifies it after the padding bit is removed. A final check of this listing round-tripped six inputs of different lengths exactly and flagged all 765 sampled single-nucleotide substitutions. One limit is worth stating: a change that flips only the padding bit of the last nucleotide leaves the checksummed bits unchanged and would not be flagged. A production version could include the padding bit in the checksum.